

Trabajo Práctico Integrador

Redes y Comunicación

Implementación, red y despliegue del proyecto final

Materia: Redes y Comunicación

Profesor: Fernando Rossler

Proyecto Final del Grupo: Sistema de Gestión Agrónoma "Don Elio"

Alumnos: Agostina Landra, Ezequiel Martínez, Martín Junges

Entrega Parcial: Parte 1 y Parte 2 — 1er Cuatrimestre

Fecha de entrega: 8 de octubre de 2026

1. Introducción

"Don Elio" es un sistema de gestión agrónoma desarrollado como una aplicación nativa para Android, destinado a profesionales del sector agropecuario. La solución resuelve la necesidad concreta de un ingeniero agrónomo o productor rural de centralizar y digitalizar el seguimiento de sus actividades productivas, reemplazando el uso tradicional de planillas físicas y cuadernos de campo.

El problema que resuelve: En la actualidad, los productores agropecuarios gestionan campañas agrícolas que involucran múltiples dimensiones complejas —insumos, cosechas, tareas operativas y observaciones de campo— de manera frecuentemente desorganizada y sin una trazabilidad clara. Don Elio unifica todas estas dimensiones en una sola herramienta, permitiendo analizar el rendimiento económico de cada campaña, gestionar tareas de forma eficiente y tomar decisiones informadas apoyadas en datos precisos.

Tipo de organización destinataria: El sistema está pensado para empresas agropecuarias de pequeña y mediana escala, cooperativas rurales y estudios de ingeniería agronómica que administran múltiples clientes.

Alcance de la solución:

- Gestión completa del ciclo de vida de campañas agrícolas (creación, seguimiento continuo y cierre).
- Registro y administración de insumos mediante un catálogo reutilizable, con vinculación directa a cada campaña.
- Planificación de tareas operativas, incluyendo recordatorios y confirmación de ejecución.
- Registro detallado de cosechas (discriminando entre lo almacenado y lo vendido/reservado) y cálculo automático de rendimiento.
- Registro de observaciones de campo con soporte para evidencia fotográfica.
- Generación de reportes comparativos entre campañas, con opción de exportación a formatos CSV y PDF.
- Autenticación segura por usuario, implementando hash de contraseñas mediante el algoritmo SHA-256.
- Mecanismos de backup y restauración de la base de datos local para salvaguardar la información.

Parte 1: Configuración de la Red e Infraestructura de Implementación

En esta sección se diseña la infraestructura de red necesaria para alojar y poner en funcionamiento el sistema "Don Elio" contemplando distintos escenarios reales. La propuesta abarca la forma en que se implementaría la solución considerando los recursos físicos disponibles, la cantidad de usuarios, los requerimientos de seguridad, las estrategias de mantenimiento y la viabilidad de crecimiento a futuro.

1.1. Diseño y Topología de la Red

Elección de topología: Estrella con Switch Central

Para implementar Don Elio en un establecimiento agropecuario estándar, se propone una **topología en estrella**. En esta configuración, todos los dispositivos finales se conectan a un switch central que, a su vez, se vincula al router encargado de proveer el acceso a Internet.

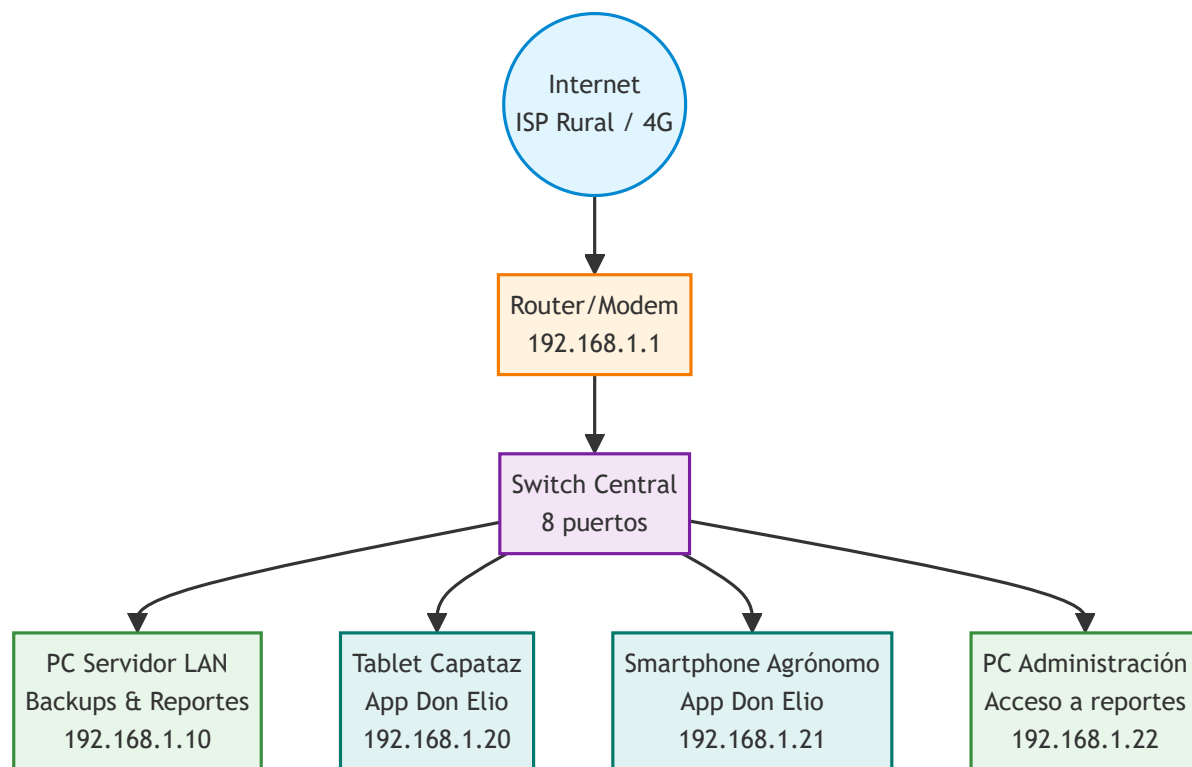
Justificación técnica para el contexto de Don Elio:

- **Aislamiento de fallas:** Si un dispositivo (como la tablet del capataz o la PC administrativa) experimenta una falla, el resto de la red no se ve afectado. Esto garantiza que el servidor de backups y reportes siga operativo, algo crítico durante la temporada de cosecha, donde la continuidad del trabajo es prioridad absoluta.
- **Escalabilidad progresiva:** En una etapa inicial, el establecimiento puede operar con tan solo 3 o 4 dispositivos. A medida que el negocio crezca y se incorporen más operarios o nuevos sectores (ej. ganadería o logística), agregar nodos adicionales consistirá simplemente en conectar nuevos cables o puntos de acceso al switch central.
- **Administración centralizada:** El uso de un switch permite monitorear el tráfico de manera eficiente. Si se utiliza un switch administrable, es posible aplicar VLANs para segmentar de forma segura la red entre los usuarios de campo y el área administrativa.
- **Bajo costo de implementación:** El hardware necesario para esta topología (un switch no administrado y un router doméstico o SOHO) es sumamente accesible, adaptándose a las posibilidades económicas de una empresa agropecuaria mediana o pequeña.

Limitación y mitigación: El switch central representa un único punto de falla (Single Point of Failure). En un escenario de empresa grande, este riesgo se mitigará implementando redundancia de enlaces y switches en configuración de cascada o apilados.

Escenario A — Cliente Pequeño (Establecimiento rural, 3–5 usuarios)

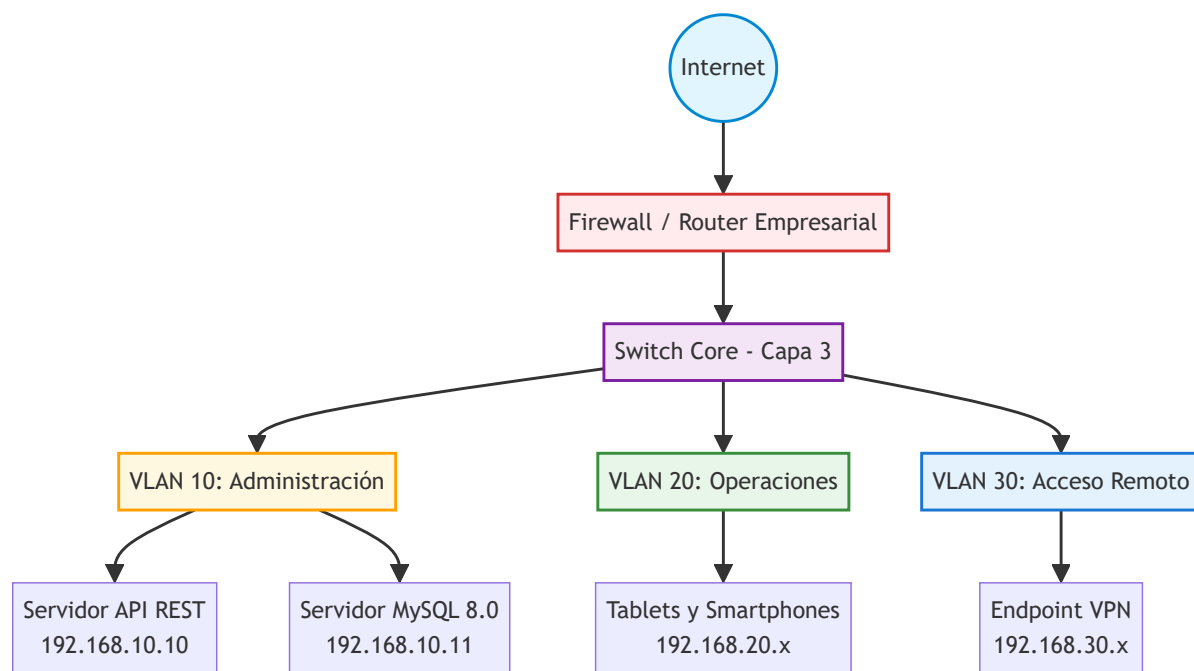
En este entorno, la aplicación Don Elio corre directamente en los dispositivos Android de los operarios de forma *offline-first* (priorizando el funcionamiento sin conexión). El "servidor" no procesa lógica compleja; es simplemente una PC o un NUC (computadora de formato pequeño y bajo consumo) que actúa como servidor de archivos para almacenar centralmente los backups de la base de datos SQLite, y como un servidor web liviano para visualizar en el futuro un panel de reportes.



La base de datos (Room / SQLite) reside localmente en el dispositivo Android de cada usuario. Los backups se exportan al servidor de archivos de la LAN mediante la funcionalidad ya desarrollada (CrearBackupUseCase). La sincronización entre dispositivos no requiere tiempo real; cada operario trabaja con su copia local en el campo (donde suele no haber señal) y la información se consolida en la oficina mediante la restauración o cruce de backups.

Escenario B — Empresa Grande (Cooperativa o estudio agronómico, 20+ usuarios)

En este escenario, el sistema Don Elio evolucionaría incorporando una **API REST centralizada**. Esta API reemplazaría las bases de datos locales por una base de datos central y compartida alojada en un servidor dedicado. La topología migraría hacia una arquitectura cliente-servidor distribuida, donde la seguridad y el rendimiento se gestionan mediante segmentación por VLANs.



Cambios clave para la adaptación a Empresa Grande:

- Separación física del Servidor de Aplicación (API REST) y el Servidor de Base de Datos para evitar bloqueos por carga.
- Implementación de un balanceador de carga (Load Balancer) mediante Nginx para soportar múltiples instancias de la API y asegurar alta disponibilidad.
- Configuración de una VPN para permitir el acceso seguro a los agrónomos externos (auditores o asesores) sin necesidad de exponer el servidor principal directamente a Internet.
- Monitoreo activo de la red y uso de recursos utilizando herramientas de grado corporativo como Zabbix o Grafana.

Plan de Direccionamiento IP (Escenario A)

Se define un rango de red privada **192.168.1.0/24**. Los dispositivos críticos (como el router y el servidor de backups) cuentan con direcciones IP estáticas para asegurar su disponibilidad permanente. Los dispositivos cliente (tablets, smartphones) obtienen su IP dinámicamente vía DHCP, preferentemente con reserva fijada por dirección MAC.

Dispositivo / Servicio	Dirección IP	Máscara de Subred	Gateway	Función en Don Elio
Router / Gateway	192.168.1.1	255.255.255.0	N/A	Provisión de salida a Internet y enrutamiento.
PC Servidor LAN	192.168.1.10	255.255.255.0	192.168.1.1	Almacenamiento de archivos .db y hosteo del panel de

Dispositivo / Servicio	Dirección IP	Máscara de Subred	Gateway	Función en Don Elio
				reportes (Nginx).
Tablet Capataz (Android)	192.168.1.20	255.255.255.0	192.168.1.1	Uso de la App Don Elio para registro de tareas y cosechas en campo.
Smartphone Agrónomo (Android)	192.168.1.21	255.255.255.0	192.168.1.1	Uso de la App Don Elio para visualización integral de reportes.
PC Administración	192.168.1.22	255.255.255.0	192.168.1.1	Equipo de escritorio para la descarga y visualización de exportaciones en CSV/PDF.

¿Por qué IP estática para el servidor? Los dispositivos Android generan las copias de seguridad apuntando constantemente a la misma dirección IP compartida en la red local. Si esta dirección variara debido al DHCP, los scripts de sincronización del lado del cliente fallarían. Mantener la IP estática (192.168.1.10) garantiza que los servicios de backup y los reportes estén siempre accesibles sin necesidad de reconfigurar la aplicación en los dispositivos móviles cada día.

1.2. Configuración Teórica del Servidor o Entorno de Ejecución

Sistema Operativo: Ubuntu Server 24.04 LTS

Justificación: Es un sistema operativo gratuito que no implica costos de licencia. Ofrece soporte a largo plazo (LTS) hasta el año 2029, brindando estabilidad sin requerir actualizaciones disruptivas. Posee compatibilidad nativa absoluta con herramientas clave como Node.js, SQLite, MySQL y Nginx. Además, su consumo de recursos es tan bajo que permite ser instalado en hardware modesto de la finca, como una Raspberry Pi 4 o una Mini PC Intel (NUC). (Nota: Para el escenario más básico de cliente pequeño, también sería factible utilizar Windows 10/11 Pro con entornos emulados como XAMPP o Laragon).

Servidor Web: Nginx

En la versión actual de Don Elio (desarrollada como aplicación móvil autónoma), Nginx se encargará de servir de manera ligera y eficiente los paneles y reportes exportados (archivos HTML, PDF, CSV) desde el servidor a las PCs de la oficina. En el futuro escenario de empresa grande, Nginx asumirá además el rol crítico de proxy inverso y balanceador de carga frente a la API REST.

```
# Instalación conceptual en Ubuntu Server:
sudo apt update && sudo apt upgrade -y
sudo apt install nginx -y
sudo systemctl enable nginx
sudo systemctl start nginx
```

Base de Datos: SQLite (local) / MySQL 8.0 (servidor centralizado)

Actualmente, el sistema Don Elio implementa **SQLite** a través de *Room* (el ORM oficial proporcionado por Android Jetpack). En este contexto, no se requiere la instalación de un motor de base de datos independiente en el servidor: la información reside íntegramente en un archivo `.db` contenido dentro de cada dispositivo móvil. Room gestiona la creación de esquemas, migraciones (actualmente en la versión 5) y relaciones complejas (Claves Foráneas) de forma automática. El archivo `.db` es el que se exporta y resguarda mediante la funcionalidad de backup.

Para la transición al escenario de **empresa grande**, con necesidad de escrituras concurrentes intensivas y mayor volumen de datos históricos, SQLite sería reemplazado íntegramente por **MySQL 8.0**, un gestor relacional robusto diseñado para entornos de red que aporta un manejo superior de transacciones, copias de seguridad en caliente y herramientas avanzadas de administración de usuarios.

Variables de Entorno y Configuración

Al migrar a la API REST, por cuestiones de seguridad, las credenciales jamás deben almacenarse (hardcodearse) en el código fuente. Los siguientes parámetros se aislarán utilizando archivos de entorno (ej. `.env`):

```
# Configuración de la API REST (escenario empresa grande)
DB_HOST=192.168.1.11
DB_PORT=3306
DB_NAME=donelio_produccion
DB_USER=donelio_app
DB_PASSWORD=<contrasena_segura>
JWT_SECRET=<clave_aleatoria_256_bits>
BACKUP_DIR=/var/backups/donelio
```

Diseño de Datos — Entidades Principales de Don Elio

A continuación, se detallan las estructuras de las tablas principales, derivadas directamente de la capa persistente (`data/local/entity/`) del código de la aplicación, modeladas originalmente en Room:

```
-- Tabla campañas: Entidad central del sistema
CREATE TABLE campañas (
```

```

    id_campania INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    fecha INTEGER NOT NULL, -- Timestamp en milisegundos para soporte universal
    cultivo TEXT NOT NULL,
    esta_activa INTEGER NOT NULL DEFAULT 1 -- 1=activa, 0=cerrada
);

-- Tabla insumos: Catálogo de insumos agrícolas predefinidos y personalizables
CREATE TABLE insumos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nombre TEXT NOT NULL,
    categoria TEXT NOT NULL, -- 'Semilla', 'Fertilizante', 'Herbicida'
    icono TEXT, -- Referencia en texto al icono Material utilizado en la UI
    activo INTEGER NOT NULL DEFAULT 1
);

-- Tabla campania_insumos: Relación N:M entre campañas e insumos, guardando precio
histórico
CREATE TABLE campania_insumos (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    id_campania INTEGER NOT NULL,
    id_insumo INTEGER NOT NULL,
    cantidad REAL NOT NULL,
    precio REAL NOT NULL,
    FOREIGN KEY (id_campania) REFERENCES campanias(id_campania) ON DELETE CASCADE,
    FOREIGN KEY (id_insumo) REFERENCES insumos(id) ON DELETE RESTRICT
);

-- Nota técnica de seguridad: La restricción ON DELETE RESTRICT en la clave foránea
-- del insumo previene a nivel de base de datos la eliminación accidental de un
-- insumo base del catálogo si este ya se encuentra asociado a una campaña activa.

```

1.3. Simulación de Pruebas de Conectividad y Funcionamiento

A continuación se detallan las metodologías y comandos que se ejecutarían para verificar la solidez de la red y del sistema en un entorno físico de implementación real.

Prueba 1 — Ping: Verificación de conectividad entre cliente y servidor

Propósito: Confirmar a nivel de red (Capa 3 del modelo OSI) que la tablet de un operario de campo o capataz alcanza correctamente el servidor de la red LAN, paso previo indispensable para el envío de backups o consultas al panel.

```
ping 192.168.1.10

# Salida esperada:
PING 192.168.1.10 56(84) bytes of data.
64 bytes from 192.168.1.10: icmp_seq=1 ttl=64 time=1.23 ms
64 bytes from 192.168.1.10: icmp_seq=2 ttl=64 time=0.98 ms
--- 192.168.1.10 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss
```

Interpretación: Una pérdida de paquetes del 0% y tiempos de respuesta (latencia) inferiores a 5 ms garantizan que la red LAN inalámbrica opera sin interferencias severas y los archivos de respaldo podrán transferirse sin riesgos de corrupción.

Prueba 2 — Traceroute: Diagnóstico preciso de la ruta de los paquetes

Propósito: Asegurar que el tráfico de datos en la LAN interna cursa únicamente por el Switch (un solo salto) sin ser enrutado ineficientemente a través de la interfaz WAN del modem/router.

```
traceroute 192.168.1.10

# Salida esperada:
traceroute to 192.168.1.10, 30 hops max
 1  192.168.1.10 (192.168.1.10)  1.230 ms  0.980 ms  1.050 ms
```

Interpretación: Un trayecto de un solo salto verifica que los dispositivos pertenecen a la misma subred (máscara /24) y se comunican ágilmente en Capa 2.

Prueba 3 — Servicio web (Panel de reportes y proxy)

Desde la PC de Administración, se verifica la correcta respuesta del servicio Nginx, que alojará temporalmente los extractos de datos generados por la app móvil.

```
curl -I http://192.168.1.10
# Resultado esperado: HTTP/1.1 200 OK
```

Prueba 4 — Base de datos: Integridad del respaldo lógico

Dado que en el escenario pequeño la BD es puramente SQLite, la prueba consiste en simular la exportación del archivo .db desde la app móvil al servidor y consultarlo remotamente vía terminal de comandos allí, validando así que el archivo no haya sufrido corrupción en la transmisión.

```
sqlite3 donelio_backup_20251023.db "SELECT COUNT(*) FROM campanias;"
# Resultado esperado: La cantidad correcta de campañas registradas en el sistema
(ej: 4).
```

Prueba 5 — Multiusuario y Concurrencia

En la arquitectura actual (Escenario A), si el agrónomo y el capataz acceden a la aplicación simultáneamente desde dispositivos diferentes, no surgen problemas de concurrencia de escritura ni bloqueos de filas, ya que cada uno interactúa de forma aislada con su propia instancia local de SQLite. Además, el caso de uso LoginUseCase comprueba y mantiene las credenciales de sesión de forma absolutamente independiente en cada equipo.

Nota sobre la adaptación: En la futura transición al Escenario B (Empresa Grande), los potenciales conflictos de escritura simultánea (cuando varios operarios intenten descontar un mismo insumo al mismo tiempo) serán delegados y resueltos por el **motor transaccional de MySQL** y las validaciones de bloqueo optimista en la API REST, garantizando la consistencia de los datos unificados.

Parte 2: Desarrollo de la Aplicación — Sistema Don Elio

En esta sección se describe en profundidad la aplicación final desarrollada, exponiendo la arquitectura subyacente, la lógica de sus procesos y las decisiones técnicas implementadas a lo largo de su construcción.

Stack Tecnológico y Arquitectura

El sistema "Don Elio" ha sido construido utilizando el siguiente ecosistema de desarrollo moderno en el ecosistema Android:

Capa / Función	Tecnología Implementada	Justificación del Rol en el Proyecto
Lenguaje	Kotlin 2.x	Lenguaje oficial y principal recomendado por Google para el desarrollo Android. Elimina NullPointerExceptions y reduce el código redundante.
Interfaz (UI)	Jetpack Compose + Material 3	Proporciona un modelo de interfaz de usuario puramente declarativa y reactiva, evitando la verbosidad y fragilidad estructural de los layouts XML clásicos.
Persistencia (BD)	Room (SQLite)	La plataforma Android ya incluye SQLite. Room añade seguridad de tipado estricta, comprobación de queries SQL en tiempo de compilación y soporte fluido para migraciones de esquema.
Inyección de Dependencias	Dagger-Hilt 2.x	Reduce drásticamente el código boilerplate para la gestión de dependencias y se integra de manera nativa con el ciclo de vida de ViewModels y Navigation Compose.
Concurrencia	Coroutines + Flow	Habilitan la ejecución asíncrona de manera legible (sin callbacks). Permiten que la UI reaccione instantáneamente ante modificaciones en la base de datos (patrón observador reactivo).
Manejo de Tareas	WorkManager	Gestiona recordatorios de tareas y sincronizaciones diferidas, garantizando su ejecución aun cuando la aplicación se encuentre cerrada o el dispositivo haya sido reiniciado.

Arquitectura del Sistema: Clean Architecture

Don Elio implementa estrictamente el patrón **Clean Architecture** (Arquitectura Limpia), promoviendo la máxima separación de responsabilidades y la alta cohesión del código fuente.

■ CAPA DE PRESENTACIÓN (Presentation)

→ Contiene las pantallas (Screens) construidas en Jetpack Compose y sus ViewModels asociados.

→ Responsable únicamente de gestionar la Interfaz de Usuario y observar los cambios de estado (StateFlow).

▼ invoca a ▼

■ CAPA DE DOMINIO (Domain)

→ Contiene los Modelos de Negocio puros y los Casos de Uso (Use Cases).

→ Totalmente agnóstica de Android (no contiene referencias a Context, Activity ni librerías de terceros).

→ Concentra la lógica de negocio central (ej. validaciones de negocio en más de 35 Use Cases documentados).

▼ implementa interfaces expuestas por ▼

■ CAPA DE DATOS (Data)

→ Administra el acceso a la Base de Datos mediante Repositorios concretos y DAOs (Data Access Objects).

→ Contiene la configuración de Room y sus entidades de tabla.

→ Contiene conversores (Mappers) que traducen entidades de persistencia a modelos del dominio.

Patrón de Flujo de Datos Unidireccional (UDF):

El flujo de la información sigue una única dirección para facilitar el seguimiento de errores, los tests unitarios y asegurar la predictibilidad del estado:

Acción en la UI Screen → Evento procesado en ViewModel → Ejecución de Lógica en el Use Case → Petición a Interfaz del Repositorio → DAO efectúa los cambios en BD SQLite.

El retorno se realiza propagando automáticamente la nueva información a través de Flow<Resource<T>>, actualizando la pantalla reactivamente sin intervención manual.

Módulos Principales y Usuarios

Módulo Principal	Responsabilidad Operativa	Casos de Uso Clave (Use Cases)
Autenticación	Verificación segura de ingreso y mantenimiento del estado de sesión activa.	LoginUseCase, RegistroUseCase
Campañas	Gestión integral del ciclo agrícola, desde su apertura hasta la liquidación final.	CrearCampaniaUC, FinalizarCampaniaUC
Insumos	Mantenimiento del catálogo de agroquímicos base y su vinculación específica con campañas.	CrearInsumoCatalogoUC, AsignarInsumoACampaniaUC
Tareas	Programación, monitoreo y notificación diferida de labores operativas.	CrearTareaUC, ConfirmarTareaUC
Cosechas	Registro de volúmenes de grano extraídos, discriminando ingresos de ventas frente a acopio.	RegistrarCosechaUC, RegistrarCosechaConVentaUC

Gestión de Permisos y Roles en Don Elio:

Actualmente el sistema cuenta con dos tipos de perfiles de acceso. El **Usuario Registrado** (el agrónomo, propietario o capataz), que ingresa sus credenciales de manera presencial, posee acceso administrativo total a la herramienta offline (permisos CRUD completos para gestionar campañas, inventario de insumos, exportación de balances y backups). Por otro lado, existe un rol de **Usuario Invitado**, habilitado por defecto únicamente en las compilaciones de desarrollo (debug builds) para facilitar el testing, el cual posee un perfil de acceso de sólo lectura sin requerir validación de credenciales.

Las contraseñas de los usuarios jamás se persisten en texto plano; estas son sometidas a una función criptográfica de dispersión irreversible (hash SHA-256) antes de ser insertadas en la base de datos, garantizando protección técnica ante intrusiones a nivel de archivo.

Diseño de la Interfaz y Experiencia de Usuario

A continuación se exponen las capturas reales representativas de las pantallas principales del sistema, que evidencian el diseño materializado, reactivo y moderno utilizando los componentes de Jetpack Compose (Material Design 3).

Vista 1 — Dashboard / Pantalla Principal:

Permite acceder rápidamente a la información más relevante del negocio agrícola de un solo vistazo, destacando la campaña principal actualmente en curso, métricas visuales y accesos rápidos a labores pendientes (Tareas).



Figura 1: Dashboard y métricas de bienvenida del sistema nativo.

Vista 2 — Detalle de Campaña y Navegación:

Centraliza exhaustivamente todas las entidades asociadas de una campaña agrícola específica (Insumos, Tareas, Cosechas, Observaciones) a través de un menú de navegación intuitivo de acceso inmediato.



Figura 2: Vista integral del estado, configuración y progreso de la campaña agrícola.

Vista 3 — Pantalla de Reportes Económicos:

Brinda herramientas analíticas visuales (gráficos de torta YCharts interactivos) y resúmenes financieros claros sobre los costos incurridos frente al volumen efectivamente producido, con opción de exportación instantánea en formatos PDF y CSV.



Figura 3: Análisis estadístico e interpretación económica (Dashboard de Inteligencia de Negocio) de la campaña.

Lógica de la Aplicación y Pseudocódigo Clave

Funcionalidad 1 — Crear Campaña

Parámetros recibidos: nombre: String, cultivo: String, fechaInicio: Long

Retorno: Flow<Resource<Unit>> (Emite progresivamente: Estado de carga, y luego Éxito o Error de negocio)

```
FUNCION crearCampaña(nombre, cultivo, fechaInicio)
  SI el nombre de la campaña está vacío ENTONCES
    EMITIR Resource.Error("El nombre no puede estar vacío, asigne un
    identificador")
    TERMINAR PROCESO
  FIN SI

  -- Construcción en memoria del objeto de Dominio
  campania = Campania(
    id = 0, -- El ORM Room se encarga de asignar el ID Auto-Incremental real
    nombre = nombre.trim(),
    cultivo = cultivo.trim(),
    fechaInicio = fechaInicio,
    estaActiva = true
  )

  EMITIR Resource.Loading -- Notifica instantáneamente a la UI que inició un proceso
  intensivo

  EN HILO DE BACKGROUND (IO Dispatcher seguro):
    campaniaRepository.insertCampaña(campania) -- El DAO ejecuta el comando SQL
    (INSERT INTO)
    EMITIR Resource.Success(Unit) -- Notifica conclusión exitosa para cerrar
    formularios
  FIN FUNCION
```

Funcionalidad 2 — Autenticación de Usuario (Login Seguro)

Parámetros recibidos: nombreUsuario: String, contraseña: String

Retorno: Usuario? (Retorna el modelo Usuario validado, o *null* si las credenciales son incorrectas)

```
FUNCION login(nombreUsuario, contraseña)
  SI nombreUsuario vacío O contraseña vacía ENTONCES
    LANZAR Excepción_Illegal("Ambos campos de acceso son obligatorios")
  FIN SI

  -- Aplicación estricta de función criptográfica Hash (Algoritmo SHA-256)
  hashIngresado = SHA256(contraseña.toByteArray(UTF-8))
```

```

-- Recuperación asíncrona del registro de usuario almacenado
usuarioEntity = usuarioDao.getUsuarioByNombre(nombreUsuario.trim())

SI usuarioEntity es null O el hash almacenado en la DB != hashIngresado ENTONCES
    DEVOLVER null -- Credenciales rechazadas, se previene el acceso
SINO
    DEVOLVER usuarioEntity.toDomain() -- Autenticación correcta, retorna modelo
mapeado
    FIN SI
FIN FUNCION

```

Manejo de Errores e Integridad Implementado:

Escenario de Error Potencial y Origen	Comportamiento Adaptativo y Respuesta del Sistema UI
Campo obligatorio vacío (Validación a nivel de Casos de Uso del Dominio)	El Caso de Uso intercepta la anomalía y emite un <code>Resource.Error</code> con texto descriptivo legible. La vista Jetpack Compose captura este estado e inyecta un texto rojo indicador de error debajo del <code>OutlinedTextField</code> específico afectado.
Intento de eliminación de un Insumo asociado (Fallo de Integridad a nivel del motor SQLite)	El motor de base de datos detecta la violación de integridad referencial estructurada (<code>FOREIGN KEY ON DELETE RESTRICT</code>) y lanza una excepción dura. El <code>ViewModel</code> captura y envuelve este crash previniendo el cierre abrupto, y presenta un <code>Snackbar</code> emergente avisando al operario que no es factible borrar el insumo del catálogo por encontrarse atado a un uso histórico de una campaña.
Negativa a otorgar permiso de la cámara (Error de acceso a Hardware/Capa OS)	La gestión reactiva de estado (<code>EstadoPermisoCamara</code>) detecta la negativa de hardware impuesta por Android. Ante esto, si se torna crítica para observar evidencia en campo, se muestra un diálogo explicativo no invasivo indicando las razones del requerimiento e invitándolo vía botón directo a la pantalla nativa de "Ajustes del Sistema" (<code>Intent Options</code>).

Aspectos Transversales: Implementación, Escalabilidad y Mantenimiento

Además de la ejecución en código, el análisis global de un sistema exige entender cómo interactuará con el tejido operativo y tecnológico de una compañía real a lo largo de los años.

Instalación y Despliegue Inicial (Cliente Pequeño)

Al encontrarnos frente a una aplicación netamente móvil y autónoma, la implantación corporativa en un cliente primario (un agricultor propietario con hasta 5 operarios) se ejecuta directamente en los dispositivos portátiles provistos (hardware corporativo sugerido: Android 8.0, 2 GB a 4 GB RAM, 100 MB de almacenamiento).

- **Proceso Físico:** Instalación controlada a través del archivo binario `app-release.apk` activando provisionalmente las "Fuentes Desconocidas" (Side-loading) en el entorno Android, o mediante un canal cerrado corporativo provisto por *Google Play Store* (Internal Testing).
- **Configuración Cero Operativa:** No se requiere asignar costosos consultores IT para configurar servidores web, firewalls perimetrales, dominios o VPNs para la operatividad de campo básica de la app. Durante la primera apertura local, el ORM *Room* instancia y materializa la base de datos de manera invisible, permitiendo crear en 20 segundos el primer usuario administrador maestro.

Estrategia de Escalabilidad (Implementación en Empresa Grande)

Con miras al futuro crecimiento para acoplar las necesidades de un grupo corporativo o cooperativa masiva (20+ usuarios interactuando concurrentemente), la actual arquitectura limitante *offline-first* presenta importantes cuellos de botella para el intercambio veloz de información estadística de las campañas. Las soluciones planteadas incluyen:

- **Desacoplamiento Base:** Migración del repositorio de persistencia (Room) hacia una **API REST externa centralizada** (desarrollada idealmente en el framework Spring Boot de Java, o Node.js) acompañada de un motor SQL potente orientado a conexiones masivas (**MySQL 8.0**). En tal circunstancia, la aplicación Android actual prescindiría de Room local y pasaría a interactuar puramente como un "cliente HTTP liviano" delegando todo procesamiento pesado de datos.
- **Autenticación Distribuida:** Transición del sistema de seguridad de aislamiento de sesión interno (verificación de hashes) hacia un estándar corporativo utilizando firmas por **JSON Web Tokens (JWT)**. Esto garantiza la posibilidad de manejar accesos controlados desde oficinas centrales (revocación de cuentas de operarios despedidos, auditorías) y configurar caducidades automáticas (time-to-live) de los logueos en tablets.

Backups, Recuperación e Integridad

En el presente, la seguridad documental de la información depende de la exportación voluntaria y metódica del archivo central .db. La aplicación contiene funcionalidad específica e integrada (CrearBackupUseCase) para extraer esta unidad de archivo compacta al invocar el marco *Storage Access Framework (SAF)* del OS, permitiendo derivarlo a un PC de resguardo o pendrive.

Al escalar el entorno hacia un servidor, se mitiga el riesgo del "olvido humano" mediante tareas desatendidas de sistema operativo (*Cron Jobs* en Linux) que efectúen diariamente un copiado profundo a nivel transaccional en la madrugada (*mysqldump*) y apliquen lógicas de compresión GZIP, borrando archivos históricos superiores a los 30 días para evitar el colapso del volumen de disco y optimizar el almacenamiento.

Mantenimiento y Auditoría

Como política esencial de mantenimiento preventivo, se integrará el SDK de una plataforma líder en analítica de estabilidad de aplicaciones móviles (ej. **Firebase Crashlytics** o Sentry). Estas herramientas se conectan silenciosamente y, ante cualquier caída inesperada en la tablet del cliente provocada por anomalías físicas (Crash o Exception no capturada), recolectan de inmediato los rastros de la pila de memoria (*Stack Trace*) y los transmiten a un portal web central al recuperar conectividad de red, permitiendo al equipo de ingeniería diagnosticar y enviar correcciones y parches actualizando el APK en cuestión de horas, sin requerir reportes telefónicos manuales.

Conclusión

El desarrollo y fundamentación técnica en red del sistema "Don Elio" demuestra fehacientemente cómo un proyecto final de índole académico se ha concebido y documentado emulando la rigurosidad técnica que demanda un ecosistema de software productivo para el mercado real.

A lo largo de este extenso análisis y despliegue operativo, se ha logrado:

- Diagramar, proyectar y validar una **infraestructura de conectividad y redes sustentable y escalable**. La transición y planeamiento detallado —partiendo desde una sencilla red en estrella acotada basada en el procesamiento táctico offline y un servidor LAN local pasivo, hacia la proyección de una arquitectura robusta orientada a servicios distribuidos (API REST) segmentada de manera segura por VLANs perimetrales— ratifica la versatilidad de los criterios de implementación adoptados.
- Revalidar el inmenso rédito técnico obtenido por emplear **arquitecturas de software modernas e invariables**, primordialmente la *Arquitectura Limpia (Clean Architecture)* en amalgama con estándares directos recomendados por Google (lenguaje Kotlin nativo, UI declarativa en Compose e Inyección sólida con Hilt). La demarcación precisa y explícita de cada responsabilidad algorítmica permite adaptar a futuro, si fuese necesario, toda la capa de Base de Datos sin rescribir y penalizar el core de negocio.
- Evidenciar de manera estructurada las **rutas y lógicas de validación transaccional** y, fundamentalmente, la absorción predecible y amistosa de los flujos de errores o excepciones informáticas, lo que avala un profundo dominio de la experiencia de interacción de usuario y de los preceptos cardinales del desarrollo Frontend móvil.

El documento final conforma, en síntesis, un plano constructivo y operativo completo. Gracias a las adaptaciones de topologías teóricas y metodológicas expuestas (abarcando desde la asistencia técnica hacia un agricultor simple hasta la centralización en una extensa cooperativa agrónoma regional), queda plenamente constatado cómo la solución "Don Elio" ha sido blindada a nivel respaldo de información, asegurada mediante bases operativas modernas, estabilizada ante infortunios procedimentales y alistada minuciosamente para ser implantada con vigor, y evolucionar armónicamente dentro del complejo mercado agropecuario y tecnológico real.

Bibliografía

Android Developers. (2024). *Room persistence library - Guías oficiales de Android*. Google Inc. Recuperado de <https://developer.android.com/training/data-storage/room>

Android Developers. (2024). *Jetpack Compose documentation y material resources*. Google Inc. Recuperado de <https://developer.android.com/jetpack/compose>

Android Developers. (2024). *Hilt dependency injection reference*. Google Inc. Recuperado de <https://developer.android.com/training/dependency-injection/hilt-android>

Android Developers. (2024). *WorkManager Overview*. Google Inc. Recuperado de <https://developer.android.com/topic/libraries/architecture/workmanager>

Nginx, Inc. (2024). *Nginx official documentation and web server fundamentals*. Recuperado de <https://nginx.org/en/docs/>

Oracle Corporation. (2024). *MySQL 8.0 Reference Manual: High availability and scalability*. Recuperado de <https://dev.mysql.com/doc/refman/8.0/en/>

OWASP Foundation. (2024). *SQL Injection Prevention Cheat Sheet: Best Security Practices*. Recuperado de https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (2024). *Password Storage Cheat Sheet: Hashing Algorithms*. Recuperado de https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Tanenbaum, A. S., & Wetherall, D. J. (2011). *Computer Networks (5th ed.)*. Pearson Prentice Hall.

Ubuntu Community. (2024). *UFW - Uncomplicated Firewall Server Documentation*. Canonical Ltd. Recuperado de <https://help.ubuntu.com/community/UFW>

Anexos

Anexo A — Diagramas de Red (Topologías)

A continuación se exponen las representaciones gráficas normalizadas para la arquitectura de red y el ruteo de información de los escenarios propuestos.

Escenario de Implementación: Cliente Pequeño (Red en Estrella Local)

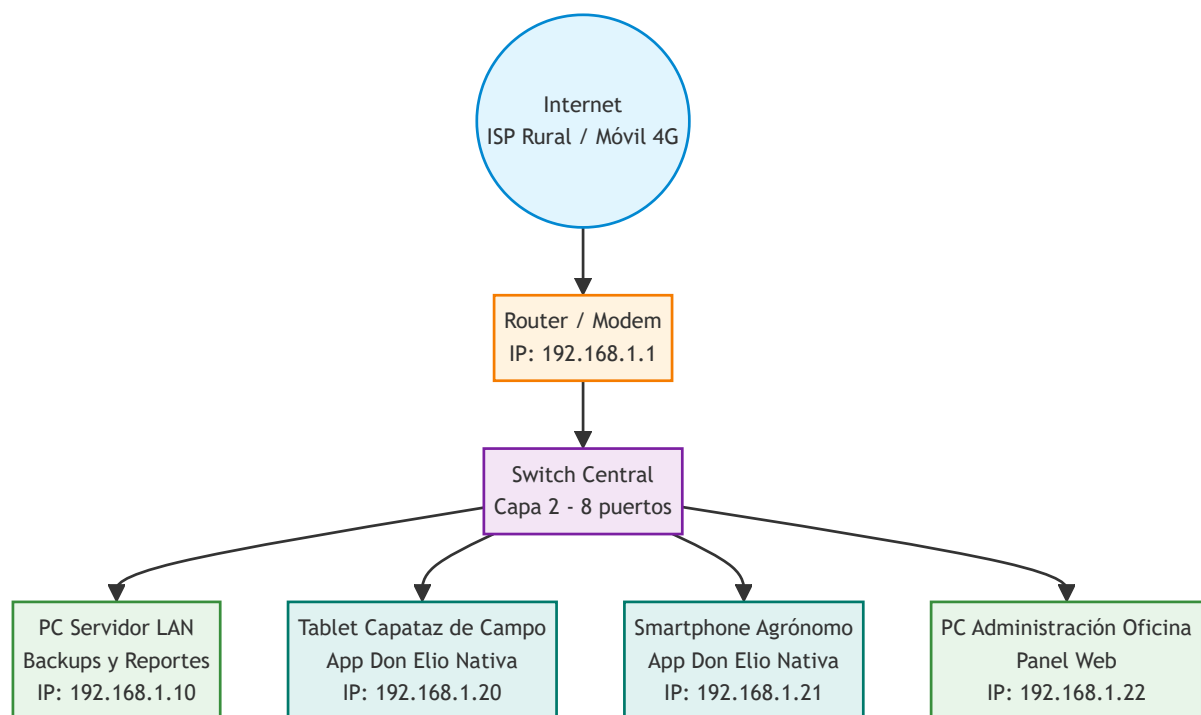


Figura 4: Topología física y conectividad para un entorno rural pequeño y autónomo.

Escenario de Implementación: Empresa Grande (Migración a API y Segmentación)

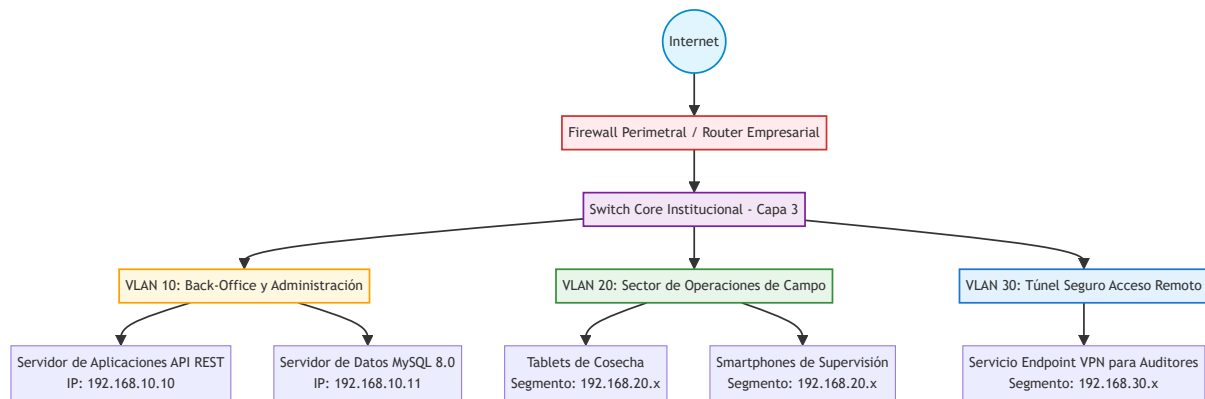


Figura 5: Propuesta de evolución a arquitectura Cliente-Servidor corporativa segmentada.

Anexo B — Fragmento de Código Fuente Real: Lógica Transaccional (CrearCampaniaUseCase.kt)

Evidencia empírica y extraída de la capa de Dominio pura, escrita formalmente en lenguaje Kotlin. Se denota la implementación de la anotación `@Inject` requerida por Hilt para la inyección de dependencias automáticas, y el empleo nativo de corrutinas Flow para emitir los estados secuenciales de un modo netamente asíncrono y reactivo a la Interfaz de Usuario.

```

class CrearCampaniaUseCase @Inject constructor(
    private val campaniaRepository: CampaniaRepository
) {
    operator fun invoke(
        nombre: String,
        cultivo: String,
        fechaInicio: Long
    ): Flow<Resource<Unit>> = flow {
        // Notificación inicial del estado "Procesando" para renderizar indicadores
        de carga visual (Loaders)
        emit(Resource.Loading)
        try {
            // Regla de Negocio Crítica: Prevención de valores anómalos o registros
            corruptos
            if (nombre.isBlank()) {
                throw IllegalArgumentException("El nombre descriptivo de la campaña
                no puede estar vacio")
            }
            // Mapeo instanciado del modelo transitorio de Dominio previo a la
            persistencia persistente
            val campania = Campania(
                id = 0, // A ser auto-incrementado en memoria por Room
                nombre = nombre.trim(),
                fechaInicio = fechaInicio,
                estaActiva = true,
                cultivo = cultivo.trim()
            )
        } catch (e: Exception) {
            emit(Resource.Error(e))
        }
    }
}
  
```

```

        )
        // Delegación y materialización a través del componente Repository
subyacente
        campaniaRepository.insertCampania(campania)

        // Confirmación exitosa y de conclusión de ciclo para liberar el UseCase
en la UI
        emit(Resource.Success(Unit))
    } catch (e: Exception) {
        // Abstracción y control centralizado del error general de runtime o IO
        emit(Resource.Error(e.localizedMessage ?: "Ocurrió un error inesperado
al procesar la campaña", e))
    }
    }.flowOn(Dispatchers.IO) // Delegación imperativa de operaciones costosas
(File/DB) fuera del Main UI Thread
}

```

Anexo C — Fragmento de Código Fuente Real: Algoritmo Criptográfico (LoginUseCase.kt)

Algoritmo de autenticación demostrando la validación estricta de variables frente a la base de datos (Data Access Object) y la generación procedimental en memoria del Hash criptográfico unívoco y estándar (SHA-256) aplicado in situ previo a cualquier comprobación o exposición a las variables temporales.

```

class LoginUseCase @Inject constructor(
    private val usuarioDao: UsuarioDao
) {
    suspend operator fun invoke(nombreUsuario: String, contrasena: String): Usuario?
    {
        // Filtrado primario e inmediato para el rechazo proactivo de credenciales
        if (nombreUsuario.isBlank()) throw IllegalArgumentException("El
identificador de usuario es mandatorio")
        if (contrasena.isBlank()) throw IllegalArgumentException("La credencial de
autenticacion esta vacia")

        // Conversión criptográfica nativa sin el uso de librerías externas
superfluas
        val hash = hashContrasena(contrasena)

        // Petición transaccional concurrente y asíncrona hacia el DAO de SQLite
        val entity = usuarioDao.getUsuarioByNombre(nombreUsuario.trim())

        // Retorno condicional estricto: la entidad debe existir y la firma Hash de
la DB debe ser correlativa
        return if (entity != null && entity.contrasena == hash)
            entity.toDomain() else null
    }
}

```

```
}

private fun hashContrasena(contrasena: String): String {
    // Implementación central del estándar Message Digest SHA-256
    val digest = MessageDigest.getInstance("SHA-256")
    val bytes = digest.digest(contrasena.toByteArray(Charsets.UTF_8))
    // Concatenación optimizada aplicando máscara y formateo hexadecimal
    uniforme a las cadenas (Byte Array)
    return bytes.joinToString("") { "%02x".format(it) }
}

}
```